Bounds Consistency

A binary constraint C on the variables X and Y with domains D_X and D_Y is *bounds consistent* if:

1. $(\exists b \in \min(D_Y) .. \max(D_Y))\,((\min(D_X), b) \in C)$, and
   $(\exists b \in \min(D_Y) .. \max(D_Y))\,((\max(D_X), b) \in C)$,
2. $(\exists a \in \min(D_X) .. \max(D_X))\,((a, \min(D_Y)) \in C)$, and
   $(\exists a \in \min(D_X) .. \max(D_X))\,((a, \max(D_Y)) \in C)$,

A CSP $\langle \mathcal{C}; \mathcal{D}_{\in} \rangle$ is bounds consistent if all its binary constraints are bounds consistent.

The following CSP is consistent, bounds consistent, but it is not arc consistent:

$$\langle 2X = Y; X\ in\ \{0, 1, 2\}, Y\ in\ \{0, 4\}\rangle$$

As a matter of fact, applying (AC1) one would get:

$$\langle 2X = Y; X\ in\ \{0, 2\}, Y\ in\ \{0, 4\}\rangle$$

The two rules for bounds consistency are the following:

$$(BC1) \qquad \frac{\langle C; X \ in \ D_X, Y \ in \ D_Y \rangle}{\langle C; X \ in \ D'_X, Y \ in \ D_Y \rangle}$$

$$(BC2) \qquad \frac{\langle C; X \ in \ D_X, Y \ in \ D_Y \rangle}{\langle C; X \ in \ D_X, Y \ in \ D'_Y \rangle}$$

- $D'_X = D_X \cap (\min(\{a \in D_X : (\exists b \in \min(D_Y)..\max(D_Y))((a,b) \in C)\})..\max(\{a \in D_X : (\exists b \in \min(D_Y)..\max(D_Y))((a,b) \in C)\}))$
- $D'_Y = D_Y \cap (\min(\{b \in D_Y : (\exists a \in \min(D_X)..\max(D_X))((a,b) \in C)\})..\max(\{b \in D_Y : (\exists a \in \min(D_X)..\max(D_X))((a,b) \in C)\}))$

Rules for BC look complex. However, they can be implemented in (almost) constant time.

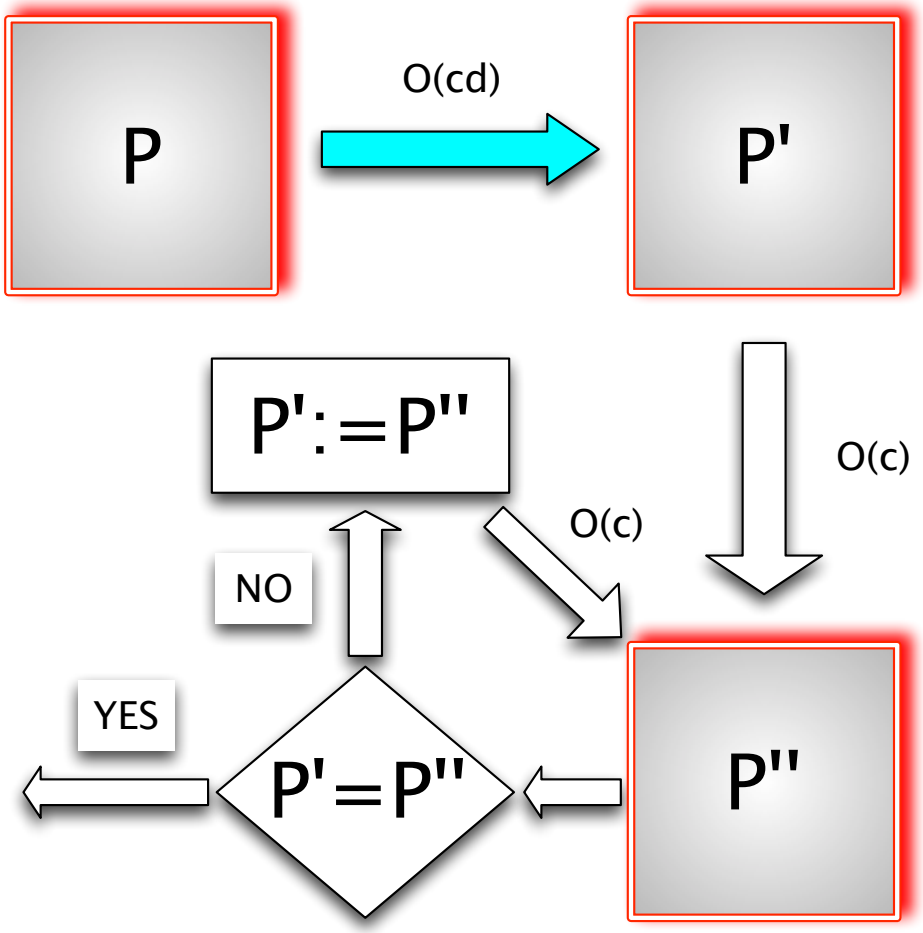
$$\frac{\langle X < Y; X\ in\ 5..20, Y\ in\ 1..10\rangle}{\langle X < Y; X\ in\ 5..9, Y\ in\ 1..10\rangle}$$

$$\frac{\langle X < Y; X\ in\ 5..9, Y\ in\ 1..10\rangle}{\langle X < Y; X\ in\ 5..9, Y\ in\ 6..10\rangle}$$

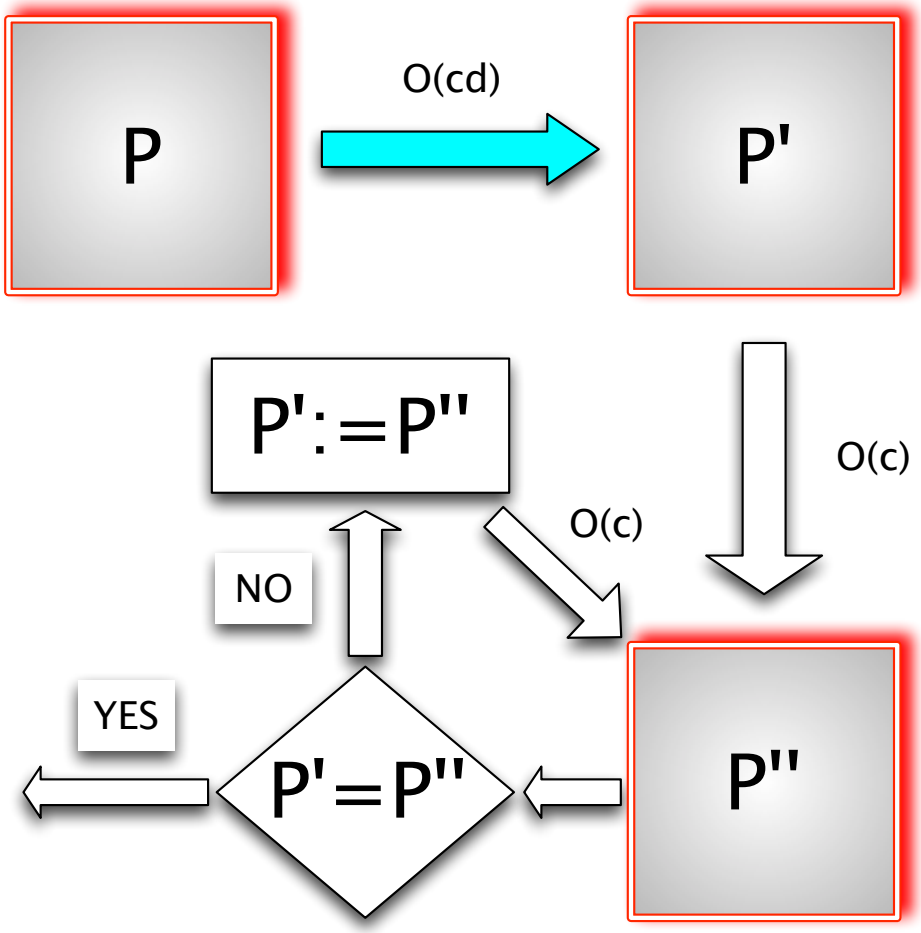
For $<$ constraints you only need to check bounds of the domains. Same idea for $=, \neq$.

Constraint Propagation

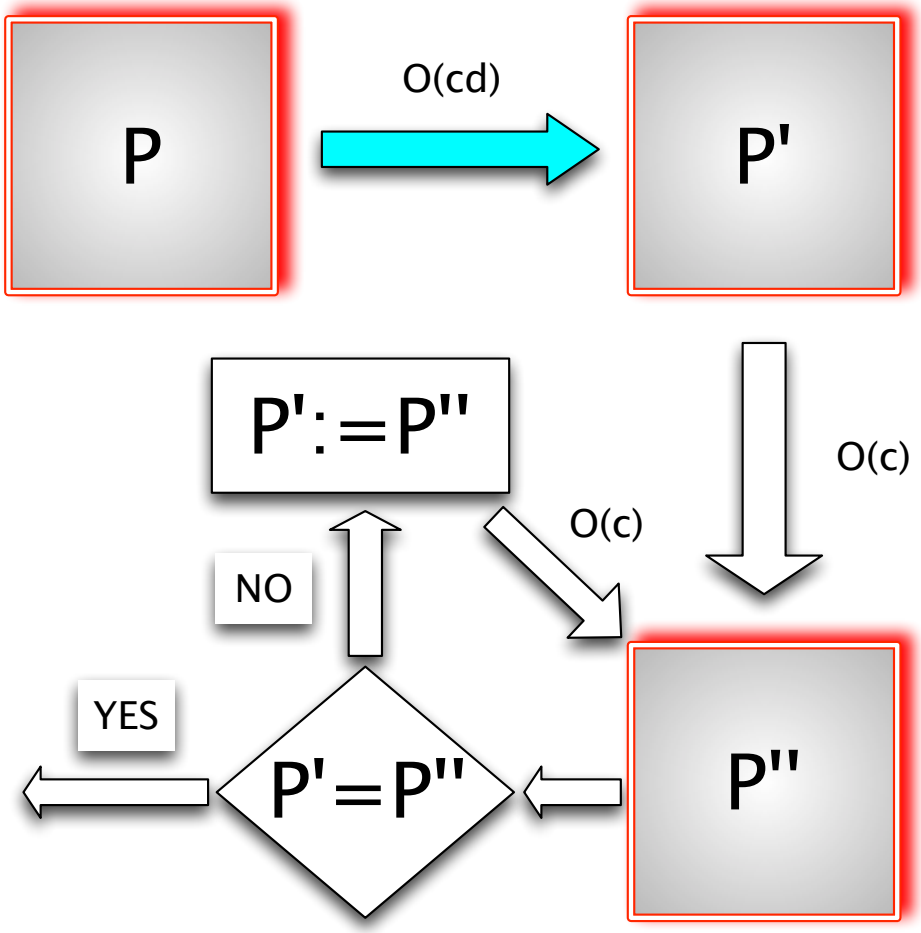
Bounds Consistency



Complexity? $O(ncd)$



Complexity? $O(ncd)$



Complexity? $O(ncd)$

- In arc/bounds we require two properties
- We could imagine a (partial) ordering between variables and consider only one of them
- This will reduce the number of loops (possibly to just one)
- In efficient implementations, each binary constraint is stored as a pair of directed binary constraints.
  As soon as, say, D_X is reduced, every constraint in which D_X appears at the left of the pair, it is added to a queue to be directionally propagated . This proves to be very efficient.
- Please have a look in literature to AC3, AC5, and AC6 propagation procedures (clever implementation of the fixpoint procedure seen above)

Non-binary constraints

It is a generalization of the arc consistency to constraints involving $m \geq 2$ variables.

HAC

A m-ary constraint C on the variables $X_1, \ldots, X_m$ is *hyper arc consistent* (HAC) is for all $i = 1, \ldots, m$ it holds that:

- $(\forall a_i \in D_i)(\exists a_1 \in D_1) \cdots (\exists a_{i-1} \in D_{i-1})(\exists a_{i+1} \in D_{i+1}) \cdots (\exists a_m \in D_m)(\langle a_1, \ldots a_m \rangle \in C)$

A CSP is *hyper arc consistent* if any constraint in it is HAC.

$$\langle 2X + 3Y < Z; X \; in \; 1..10, Y \; in \; 1..10, Z \; in \; 1..10 \rangle$$

Any pair of values for X and Y force Z to be at least 6. Thus, since $1 \in D_Z$, the CSP is not GAC.
The value 3 in D_Y is also not supported. With $Y = 3$ the expression in the l.h.s. is at least 11 (2+9).
The equivalent HAC CSP is

$$\langle 2X + 3Y < Z; X \; in \; 1..3, Y \; in \; 1..2, Z \; in \; 6..10 \rangle$$

HAC can be obtained by repatedly applying the rules ($HACi$) rules (for $i = 1, \ldots, m$):

$$\frac{\langle C; X_1 \ in \ D_1, \ldots, X_{i-1} \ in \ D_{i-1}, X_i \ in \ D_i, X_{i+1} \ in \ D_{i+1}, \ldots, X_m \ in \ D_m \rangle}{\langle C; X_1 \ in \ D_1, \ldots, X_{i-1} \ in \ D_{i-1}, X_i \ in \ D'_i, X_{i+1} \ in \ D_{i+1}, \ldots, X_m \ in \ D_m \rangle}$$

where C is a constraint on $X_1, \ldots, X_m$ and

$$\begin{aligned} D'_i \quad = \quad & \{a_i \in D_i \ : \ (\exists a_1 \in D_1) \cdots (\exists a_{i-1} \in D_{i-1}) \\ & (\exists a_{i+1} \in D_{i+1}) \cdots (\exists a_m \in D_m)(\langle a_1, \ldots \ a_m \rangle \in C)\} \end{aligned}$$

Complexity? $O(ncmd^{m+1})$

HAC can be obtained by repatedly applying the rules ($HACi$) rules (for $i = 1, \ldots, m$):

$$\frac{\langle C; X_1 \ in \ D_1, \ldots, X_{i-1} \ in \ D_{i-1}, X_i \ in \ D_i, X_{i+1} \ in \ D_{i+1}, \ldots, X_m \ in \ D_m \rangle}{\langle C; X_1 \ in \ D_1, \ldots, X_{i-1} \ in \ D_{i-1}, X_i \ in \ D'_i, X_{i+1} \ in \ D_{i+1}, \ldots, X_m \ in \ D_m \rangle}$$

where C is a constraint on $X_1, \ldots, X_m$ and

$$\begin{aligned} D'_i \quad = \quad & \{a_i \in D_i \ : \ (\exists a_1 \in D_1) \cdots (\exists a_{i-1} \in D_{i-1}) \\ & (\exists a_{i+1} \in D_{i+1}) \cdots (\exists a_m \in D_m)(\langle a_1, \ldots \ a_m \rangle \in C)\} \end{aligned}$$

Complexity? $O(ncmd^{m+1})$

HAC can be obtained by repatedly applying the rules ($HACi$) rules (for $i = 1, \ldots, m$):

$$\frac{\langle C; X_1 \ in \ D_1, \ldots, X_{i-1} \ in \ D_{i-1}, X_i \ in \ D_i, X_{i+1} \ in \ D_{i+1}, \ldots, X_m \ in \ D_m \rangle}{\langle C; X_1 \ in \ D_1, \ldots, X_{i-1} \ in \ D_{i-1}, X_i \ in \ D'_i, X_{i+1} \ in \ D_{i+1}, \ldots, X_m \ in \ D_m \rangle}$$

where C is a constraint on $X_1, \ldots, X_m$ and

$$\begin{aligned} D'_i \quad = \quad & \{a_i \in D_i \ : \ (\exists a_1 \in D_1) \cdots (\exists a_{i-1} \in D_{i-1}) \\ & (\exists a_{i+1} \in D_{i+1}) \cdots (\exists a_m \in D_m)(\langle a_1, \ldots \ a_m \rangle \in C)\} \end{aligned}$$

Complexity? $O(ncmd^{m+1})$

Advanced forms of local consistency

- A CSP $\mathcal{P}$ is *normalized* if for each pair of variables X, Y (the order is not important) there is at most one binary constraint including them in $\mathcal{P}$.
- If $\mathcal{P}$ is normalized, let us denote by $C(X, Y)$ such a constraint if it exists. Otherwise we can define $C(X, Y) = D_X \times D_Y$.
- A CSP $\mathcal{P}$ is *standard* if for each (unordered) pair of variables X, Y there is exactly one binary constraints on them (called $C(X, Y)$) in $\mathcal{P}$.

$$\langle \underbrace{X+Y<5}_{C(X,Y)}, \underbrace{X+Y\leq Z}_{C(X,Y,Z)}, \underbrace{X<Z}_{C(X,Z)}, \underbrace{2X\neq 3Z}_{C(X,Z)}, \underbrace{X+Y\neq Z}_{C(X,Y,Z)}; \mathcal{D}_{\in}\rangle$$

- It is not normalized (there are two constraints for X, Z). Anyway, we can imagine to replace C_1 and C_2 by $C_1 \cap C_2$ (constraints are sets of tuples!!!).
- If we add the constraint $D_Y \times D_Z$ (let us call it `all`) the CSP becomes standard.

$$\langle \underbrace{X+Y<5}_{C(X,Y)}, \underbrace{(X<Z \wedge 2X\neq 3Z)}_{C(X,Z)}, \underbrace{\texttt{all}(Y,Z)}_{C(Y,Z)}, \underbrace{X+Y<Z}_{C(X,Y,Z)}; \mathcal{D}_{\in}\rangle$$

The rewriting algorithm just sketched can be formalized. Any CSP can be converted to a standard and equivalent CSP. (during the rewriting stage one can detect unsatisfiability by forcing arc consistency)

$$\langle X + Y = 0, X - Y = 0; X\ in\ \{-1, 1\}, Y\ in\ \{-1, 1\}\rangle$$

is arc consistent but not standard.

$$\langle (X + Y = 0 \wedge X - Y = 0); X\ in\ \{-1, 1\}, Y\ in\ \{-1, 0, 1\}\rangle$$

is standard but not arc consistent.

- Given a binary relation R, we denote $R^T = \{(b, a) : (a, b) \in R\}$
- Given R and S binary relations, we denote:

$$RS = \{(a, b) : \exists c\,((a, c) \in R, (c, b) \in S)\}$$

PATH CONSISTENCY

A standard CSP is *path consistent* if for each triple of variables X, Y, Z it holds that

$$C(X, Z) \subseteq C(X, Y)C(Y, Z)$$

Namely, if whenever $(a, b) \in C(X, Z)$ there is a $c \in D_Y$ such that $(a, c) \in C(X, Y)$ and $(c, b) \in C(Y, Z)$.

Let us observe that implicitly it is also required that

$$C(X, Y) \subseteq C(X, Z)C(Z, Y) \text{ and } C(Y, Z) \subseteq C(Y, X)C(X, Z)$$

- Let us consider:

$$\langle X \neq Y, Y \neq Z, X \neq Z; X\ in\ 0..1, Y\ in\ 0..1, Z\ in\ 0..1 \rangle$$

- Then $C(X, Z) = C(X, Y) = C(Y, Z) = \{(0, 1), (1, 0)\}$.
- Observe that $C(X, Y)C(Y, Z) = \{(0, 0), (1, 1)\}$
- and then

$$C(X, Z) = \{(0, 1), (1, 0)\} \not\subseteq \{(0, 0), (1, 1)\} = C(X, Y)C(Y, Z)$$

- Namely it is not path consistent.

$$(PC1) \qquad \frac{\langle C(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}{\langle C'(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}$$

$$(PC2) \qquad \frac{\langle C(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}{\langle C(X,Y), C'(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}$$

$$(PC3) \qquad \frac{\langle C(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}{\langle C'(X,Y), C(X,Z), C'(Y,Z); \mathcal{D}_{\in}\rangle}$$

where:

- $C'(X,Y) = C(X,Y) \cap (C(X,Z)C(Y,Z)^T)$
- $C'(X,Z) = C(X,Z) \cap (C(X,Y)C(Y,Z))$
- $C'(Y,Z) = C(Y,Z) \cap (C(X,Y)^T C(X,Z))$

Complexity? $O(vd\, v(v-1)(v-2)\, d^4) = O(v^4 d^5)$

$$(PC1) \qquad \frac{\langle C(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}{\langle C'(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}$$

$$(PC2) \qquad \frac{\langle C(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}{\langle C(X,Y), C'(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}$$

$$(PC3) \qquad \frac{\langle C(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}{\langle C'(X,Y), C(X,Z), C'(Y,Z); \mathcal{D}_{\in}\rangle}$$

where:

- $C'(X,Y) = C(X,Y) \cap (C(X,Z)C(Y,Z)^T)$
- $C'(X,Z) = C(X,Z) \cap (C(X,Y)C(Y,Z))$
- $C'(Y,Z) = C(Y,Z) \cap (C(X,Y)^T C(X,Z))$

Complexity? $O(vd\, v(v-1)(v-2)\, d^4) = O(v^4 d^5)$

$$(PC1) \qquad \frac{\langle C(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}{\langle C'(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}$$

$$(PC2) \qquad \frac{\langle C(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}{\langle C(X,Y), C'(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}$$

$$(PC3) \qquad \frac{\langle C(X,Y), C(X,Z), C(Y,Z); \mathcal{D}_{\in}\rangle}{\langle C'(X,Y), C(X,Z), C'(Y,Z); \mathcal{D}_{\in}\rangle}$$

where:

- $C'(X,Y) = C(X,Y) \cap (C(X,Z)C(Y,Z)^T)$
- $C'(X,Z) = C(X,Z) \cap (C(X,Y)C(Y,Z))$
- $C'(Y,Z) = C(Y,Z) \cap (C(X,Y)^T C(X,Z))$

Complexity? $O(vd\, v(v-1)(v-2)\, d^4) = O(v^4 d^5)$

- It is a generalization of the previous notions.
- Let us skip it for now.

CONSTRAINT SOLVING

Let's go back to the roots: SAT solving

- As we have seen, a SAT instance can be seen as a CSP
- $D_i = \{0, 1\}$ for all variables
- Constraints are clauses

$$\ell_1 \vee \cdots \vee \ell_n$$

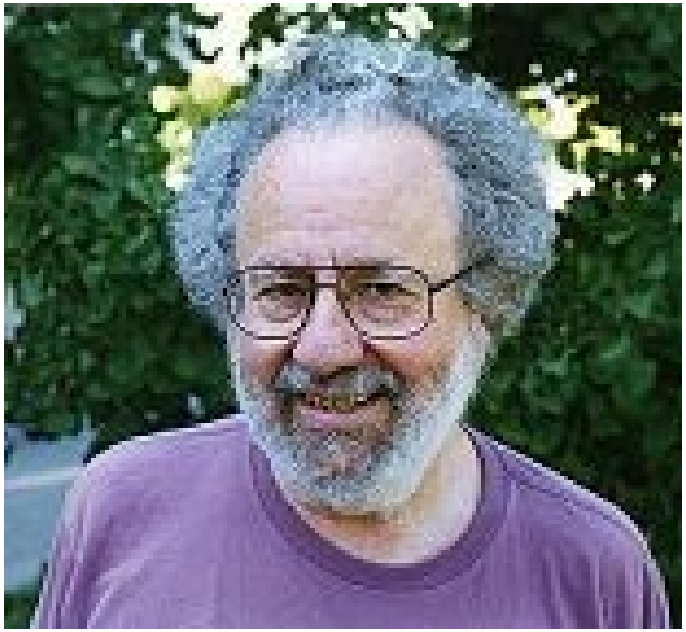
  where each ℓ_i is a variable or the negation of a variable
- Unary constraints:
  - $(X_i) \Rightarrow X_i = 1$
  - $(\neg X_i) \Rightarrow X_i = 0$

  They could simply be removed at the beginning of the process.
- Binary constraints:

$$(\ell_1 \vee \ell_2)$$

  It cannot be the case that both are false!

SAT solving



Martin Davis (1928–2023)

Hillary Putnam (1926–2016)

DP . . . (1960)

A Computing Procedure for Quantification Theory*

MARTIN DAVIS

Rensselaer Polytechnic Institute, Hartford Division, East Windsor Hill, Conn.

AND

HILARY PUTNAM

Princeton University, Princeton, New Jersey

6. *An Example*

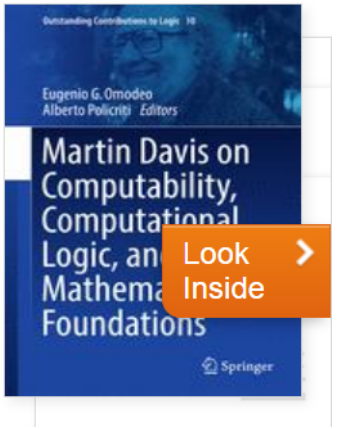
P. C. Gilmore[18] tested his refutation-procedure on a number of formulas, including the following one:

$$(Ex)(Ey)(z)\{(F(x, y) \rightarrow (F(y, z) \ \& \ F(z, z))) \ \& \ ((F(x, y) \ \& \ G(x, y)) \rightarrow (G(x, z) \ \& \ G(z, z)))\} \quad (1)$$

We have selected this example for purposes of comparison because (a) it is not so long as to make hand computation immediately impractical (e.g., it is already in prenex form, and the matrix can easily be put into conjunctive normal form); yet (b) Gilmore's procedure did *not* lead to a refutation although an IBM 704 was employed for 21 minutes.

Our procedure, on the other hand, *did* lead to a refutation in under a half-hour of *hand* computation!

[18] Cf. PAUL C. GILMORE, A proof method for quantification theory. *IBM J. Research Dev.* 4 (1960), 28–35.



DP . . . (1960)

A Computing Procedure for Quantification Theory*

MARTIN DAVIS

Rensselaer Polytechnic Institute, Hartford Division, East Windsor Hill, Conn.

AND

HILARY PUTNAM

Princeton University, Princeton, New Jersey

6. *An Example*

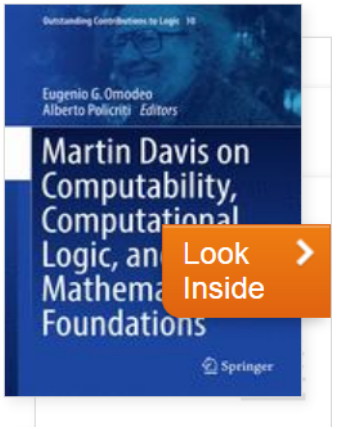
P. C. Gilmore[18] tested his refutation-procedure on a number of formulas, including the following one:

$$(Ex)(Ey)(z)\{(F(x, y) \rightarrow (F(y, z) \ \& \ F(z, z))) \ \& \ ((F(x, y) \ \& \ G(x, y)) \rightarrow (G(x, z) \ \& \ G(z, z)))\} \quad (1)$$

We have selected this example for purposes of comparison because (a) it is not so long as to make hand computation immediately impractical (e.g., it is already in prenex form, and the matrix can easily be put into conjunctive normal form); yet (b) Gilmore's procedure did *not* lead to a refutation although an IBM 704 was employed for 21 minutes.

Our procedure, on the other hand, *did* lead to a refutation in under a half-hour of *hand* computation!

[18] Cf. PAUL C. GILMORE, A proof method for quantification theory. *IBM J. Research Dev.* 4 (1960), 28–35.



DPLL —page 323 of Davis' Book

The task of implementing the DP procedure was split. Logemann undertook the parsing of the CNF formula, entered in Polish notation, the formula preparation having been done by hand. Logemann also handled the structuring of the set of clauses, while Loveland took on the testing of the clause set for consistency. The program was written in SAP, the Symbolic Assembler Program, the assembly language for the IBM 704. After the first runs, which quickly saturated the 32,768 36-bit words of available storage, George Logemann suggested that Rule III be replaced with a splitting rule. He noted that it was easy to save the current environment on tape, and retrieve it on backtracking. As for the other systems with splitting rules, this led to depth-first search. Thus, with the new program, instead of clause addition there was clause removal. Interspersed with applications of Rules I and II, the program recursed on Rule III*, saving the environment on tape for backtracking. This solved the space issue, at least until the input clause set overwhelmed the memory. Now very important, but not emphasized at the time, was the easy definition of a satisfying assignment should there be one. It is unclear why "turning off" and then "turning on" various clauses and literals on backtracking was not pursued, as writing to tape is a slow operation. Using the appropriate list structures, top level routines existed that quickly deleted all occurrences of a specified literal and eliminated the appropri-

DPLL —page 323 of Davis' Book

The task of implementing the DP procedure was split. Logemann undertook the parsing of the CNF formula, entered in Polish notation, the formula preparation having been done by hand. Logemann also handled the structuring of the set of clauses, while Loveland took on the testing of the clause set for consistency. The program was written in SAP, the Symbolic Assembler Program, the assembly language for the IBM 704. After the first runs, which quickly saturated the 32,768 36-bit words of available storage, George Logemann suggested that Rule III be replaced with a splitting rule. He noted that it was easy to save the current environment on tape, and retrieve it on backtracking. As for the other systems with splitting rules, this led to depth-first search. Thus, with the new program, instead of clause addition there was clause removal. Interspersed with applications of Rules I and II, the program recursed on Rule III*, saving the environment on tape for backtracking. This solved the space issue, at least until the input clause set overwhelmed the memory. Now very important, but not emphasized at the time, was the easy definition of a satisfying assignment should there be one. It is unclear why "turning off" and then "turning on" various clauses and literals on backtracking was not pursued, as writing to tape is a slow operation. Using the appropriate list structures, top level routines existed that quickly deleted all occurrences of a specified literal and eliminated the appropri-

DPLL —page 323 of Davis' Book

The task of implementing the DP procedure was split. Logemann undertook the parsing of the CNF formula, entered in Polish notation, the formula preparation having been done by hand. Logemann also handled the structuring of the set of clauses, while Loveland took on the testing of the clause set for consistency. The program was written in SAP, the Symbolic Assembler Program, the assembly language for the IBM 704. After the first runs, which quickly saturated the 32,768 36-bit words of available storage, George Logemann suggested that Rule III be replaced with a splitting rule. He noted that it was easy to save the current environment on tape, and retrieve it on backtracking. As for the other systems with splitting rules, this led to depth-first search. Thus, with the new program, instead of clause addition there was clause removal. Interspersed with applications of Rules I and II, the program recursed on Rule III*, saving the environment on tape for backtracking. This solved the space issue, at least until the input clause set overwhelmed the memory. Now very important, but not emphasized at the time, was the easy definition of a satisfying assignment should there be one. It is unclear why "turning off" and then "turning on" various clauses and literals on backtracking was not pursued, as writing to tape is a slow operation. Using the appropriate list structures, top level routines existed that quickly deleted all occurrences of a specified literal and eliminated the appropri-

DPLL —page 323 of Davis' Book

The task of implementing the DP procedure was split. Logemann undertook the parsing of the CNF formula, entered in Polish notation, the formula preparation having been done by hand. Logemann also handled the structuring of the set of clauses, while Loveland took on the testing of the clause set for consistency. The program was written in SAP, the Symbolic Assembler Program, the assembly language for the IBM 704. After the first runs, which quickly saturated the 32,768 36-bit words of available storage, George Logemann suggested that Rule III be replaced with a splitting rule.

and retrieve it

III *Rule for eliminating atomic formulas.* $(A \vee p) \wedge (B \vee \neg p) \wedge R$ is unsatisfiable iff $(A \vee B) \wedge R$ is unsatisfiable.

Interspersed with applications of Rules I and II, the program recursed on Rule III*, saving the environment on tape for backtracking. This solved the space issue, at least until the input clause set overwhelmed the memory. Now very important, but not emphasized at the time, was the easy definition of a satisfying assignment should there be one. It is unclear why "turning off" and then "turning on" various clauses and literals on backtracking was not pursued, as writing to tape is a slow operation. Using the appropriate list structures, top level routines existed that quickly deleted all occurrences of a specified literal and eliminated the appropri-

DPLL —page 323 of Davis' Book

The task of implementing the DP procedure was split. Logemann undertook the parsing of the CNF formula, entered in Polish notation, the formula preparation having been done by hand. Logemann also handled the structuring of the set of clauses, while Loveland took on the testing of the clause set for consistency. The program was written in SAP, the Symbolic Assembler Program, the assembly language for the IBM 704. After the first runs, which quickly saturated the 32,768 36-bit words of available storage, George Logemann suggested that Rule III be replaced with a splitting rule. He noted that it was easy to save the current environment on tape, and retrieve it on backtracking. As for the other systems with splitting rules, this led to depth-first search. Thus, with the new program, instead of clause addition there was clause removal. Interspersed with applications of Rules I and II, the program recursed on Rule III*, saving the environment on tape for backtracking. This solved the space issue, at least until the input clause set overwhelmed the memory. Now very important, but not emphasized at the time, was the easy definition of a satisfying assignment should there be one. It is unclear why "turning off" and then "turning on" various clauses and literals on backtracking was not pursued, as writing to tape is a slow operation. Using the appropriate list structures, top level routines existed that quickly deleted all occurrences of a specified literal and eliminated the appropri-

DPLL —page 323 of Davis' Book

The task of implementing the DP procedure was split. Logemann undertook the parsing of the CNF formula, entered in Polish notation, the formula preparation having been done by hand. Logemann also handled the structuring of the set of clauses, while Loveland took on the testing of the clause set for consistency. The program was written in SAP, the Symbolic Assembler Program, the assembly language for the IBM 704. After the first runs, which quickly saturated the 32,768 36-bit words of available storage, George Logemann suggested that Rule III be replaced with a splitting rule. He noted that it was easy to save the current environment on tape, and retrieve it on backtracking. As for the other systems with splitting rules, this led to depth-first search. Thus, with the new program, instead of clause addition there was clause removal. Interspersed with applications of Rules I and II, the program recursed on Rule III*, saving the environment on tape for backtracking. This solved the space issue, at least until the input clause set overwhelmed the memory. Now very important, but not emphasized at the time, was the easy definition of a satisfying assignment should there be one. It is unclear why "turning off" and then "turning on" various clauses and literals on backtracking was not pursued, as writing to tape is a slow operation. Using the appropriate list structures, top level routines existed that quickly deleted all occurrences of a specified literal and eliminated the appropri-

- Consider a clause

$$X_1 \vee \neg X_2 \vee X_3$$

  If the current assignment is $X_1 = 0, X_2 = 1$, you can deterministically infer that $X_3 = 1$.
- All clauses that are not already satisfied are checked. If all but one literal are assigned, assign it deterministically and proceed until there are not such clauses. This is a very effective form of reasoning that reduces the search space.
- There are clever techniques for implementing UP that avoid to scan of all the formula (e.g. watched literals)
- Modern SAT solvers also use (conflict driven) clause learning, heuristics, restarts, and backjumping, parallelism.

- Consider a clause

$$X_1 \vee \neg X_2 \vee X_3$$

  If the current assignment is $X_1 = 0, X_2 = 1$, you can deterministically infer that $X_3 = 1$.
- All clauses that are not already satisfied are checked. If all but one literal are assigned, assign it deterministically and proceed until there are not such clauses. This is a very effective form of reasoning that reduces the search space.
- There are clever techniques for implementing UP that avoid to scan of all the formula (e.g. watched literals)
- Modern SAT solvers also use (conflict driven) clause learning, heuristics, restarts, and backjumping, parallelism.

- Consider a clause
$$X_1 \vee \neg X_2 \vee X_3$$
If the current assignment is $X_1 = 0, X_2 = 1$, you can deterministically infer that $X_3 = 1$.
- All clauses that are not already satisfied are checked. If all but one literal are assigned, assign it deterministically and proceed until there are not such clauses. This is a very effective form of reasoning that reduces the search space.
- There are clever techniques for implementing UP that avoid to scan of all the formula (e.g. watched literals)
- Modern SAT solvers also use (conflict driven) clause learning, heuristics, restarts, and backjumping, parallelism.

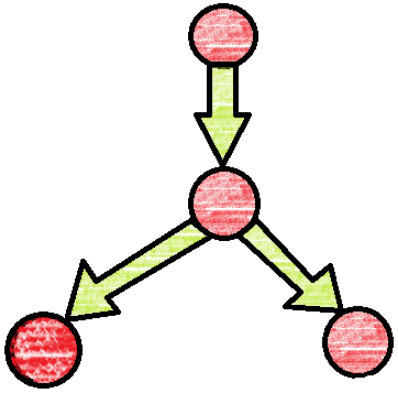
Given Φ (CNF), establishing whether exists θ s.t. $\Phi\theta$ is true.

```
DP(Φ,θ)
    θ' ← unit_propagation(Φ,θ)
    if (ok(Φθ')) return θ'
    else if (ko(Φθ')) return false
    else X ← select_variable(Φ,θ')
        θ'' ←DP(Φ, θ'[X/true])
        if (θ'' ≠ false) return θ''
        else return DP(Φ, θ'[X/false])
```

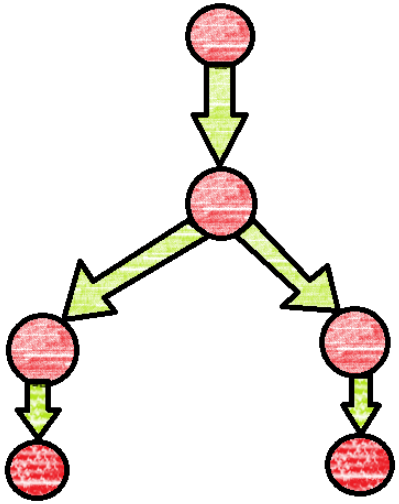
Searching=Propagation+ND assignment

Searching=Propagation+ND assignment

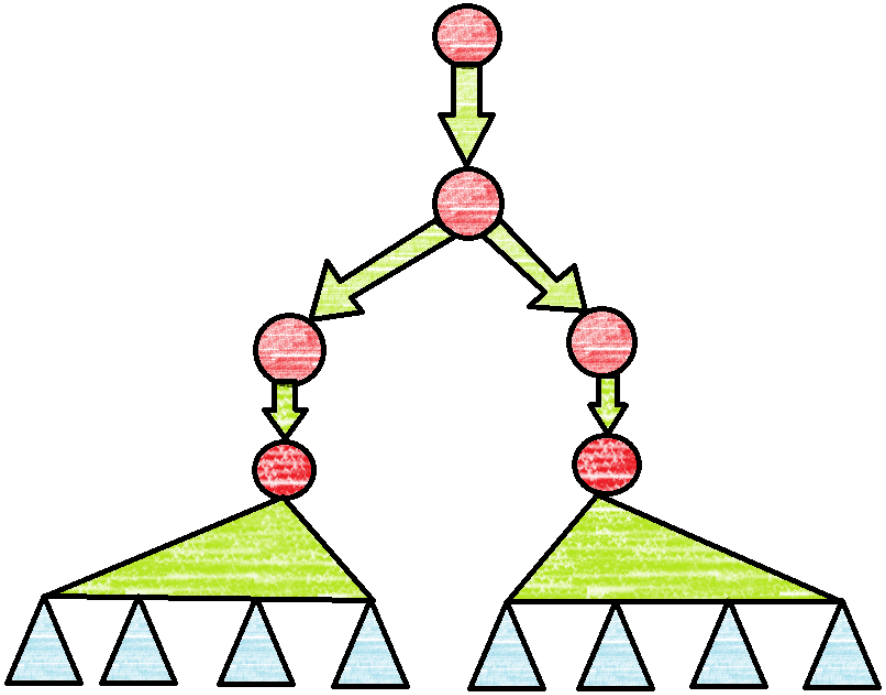
Searching=Propagation+ND assignment



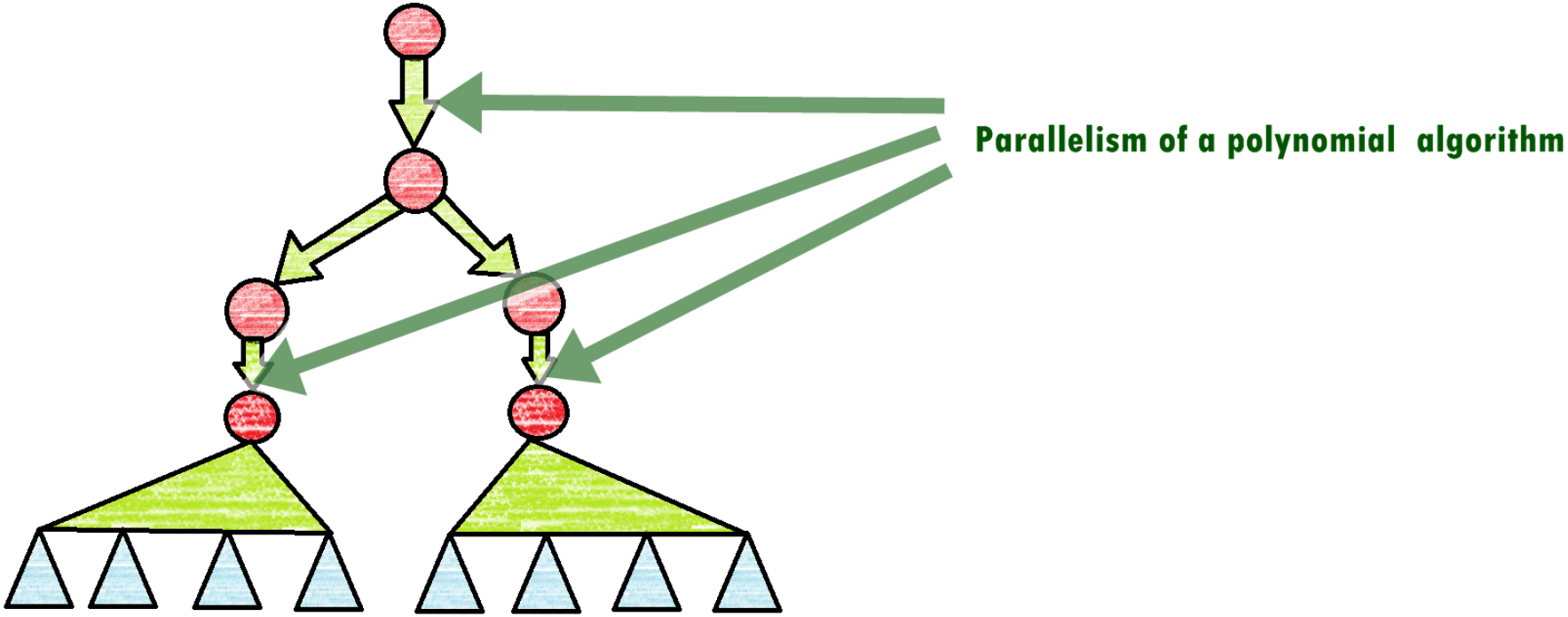
Searching=Propagation+ND assignment



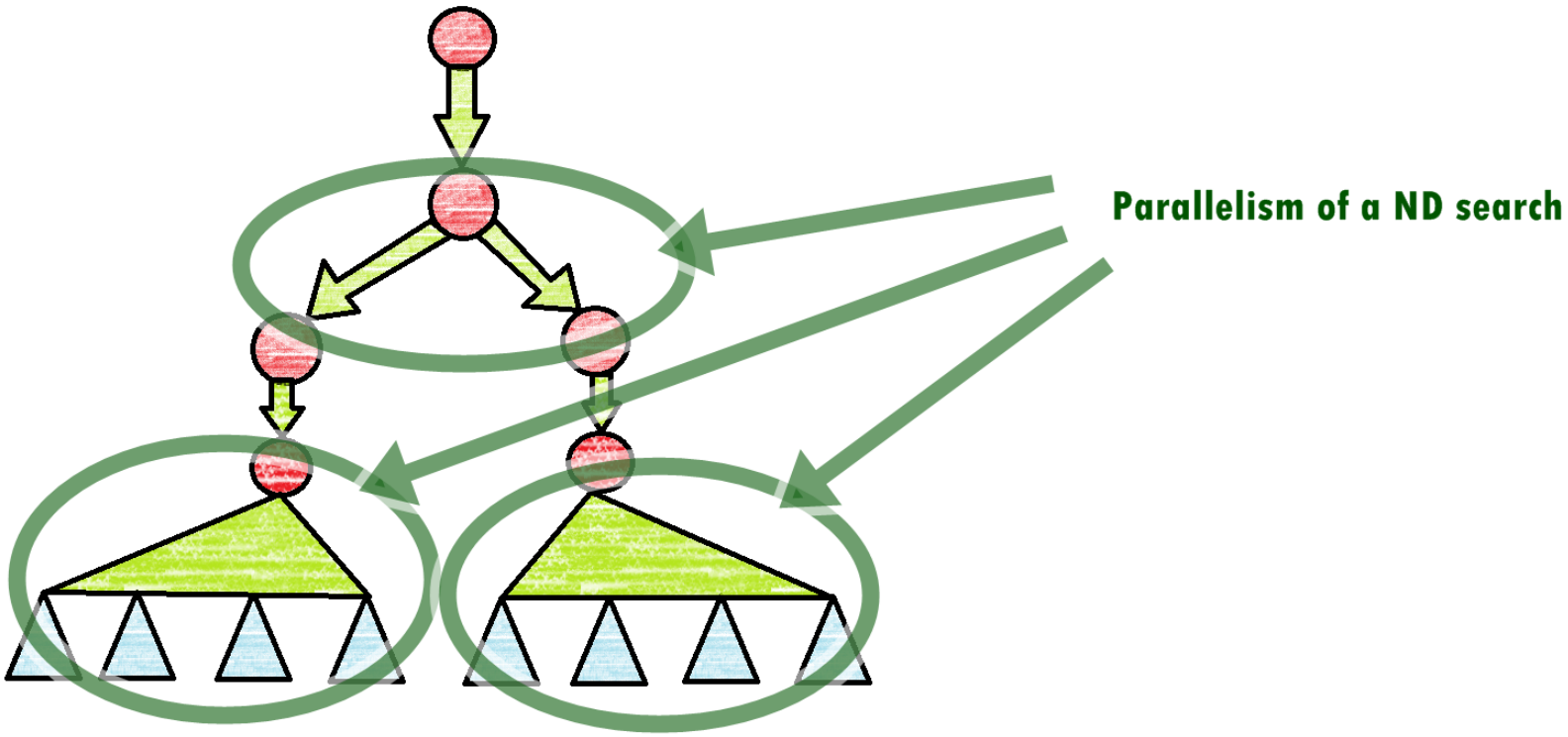
Searching=Propagation+ND assignment



Searching=Propagation+ND assignment



Searching=Propagation+ND assignment



Constraint Solving

As seen in DPLL for SAT, solution search alternates two stages: non-deterministic choices and constraint propagation (aiming at reaching some form of local consistency).

- ♡ A variable is chosen (using a suitable heuristics) and its domain is reduced (typically, by assigning it a value, but other possibilities are considered e.g., splitting the domain in two parts, etc.)
- Alternatively (less frequent in implementations) a constraint is chosen and split
- Then propagation is applied
- If a domain becomes empty, then (fail) backtracking!
- If there are no longer uninstantiated variables, the solution is returned.
- Otherwise, go to ♡

Size and shape of the search tree depend on variable selection, on the kind of propagation used, on the kind of assignment/domain reduction rules employed.

1. (domain) labeling:

$$\frac{X \in \{a_1, \ldots, a_k\}}{X \in \{a_1\} | \cdots | X \in \{a_k\}}$$

2. (domain) enumeration:

$$\frac{X \in \mathcal{D}}{X \in \{a\} | X \in \mathcal{D} \setminus \{a\}}$$

where $a \in \mathcal{D}$

3. (domain) bisection:

$$\frac{X \in \mathcal{D}}{X \in \min(\mathcal{D})..a | X \in b..\max(\mathcal{D})}$$

where $a, b \in \mathcal{D}$, and b is the element following a in $\mathcal{D}$. If $\mathcal{D}$ is an interval $x..y$ choose $a = \lfloor (x+y)/2 \rfloor$ and $b = a + 1$.

1. Implication:

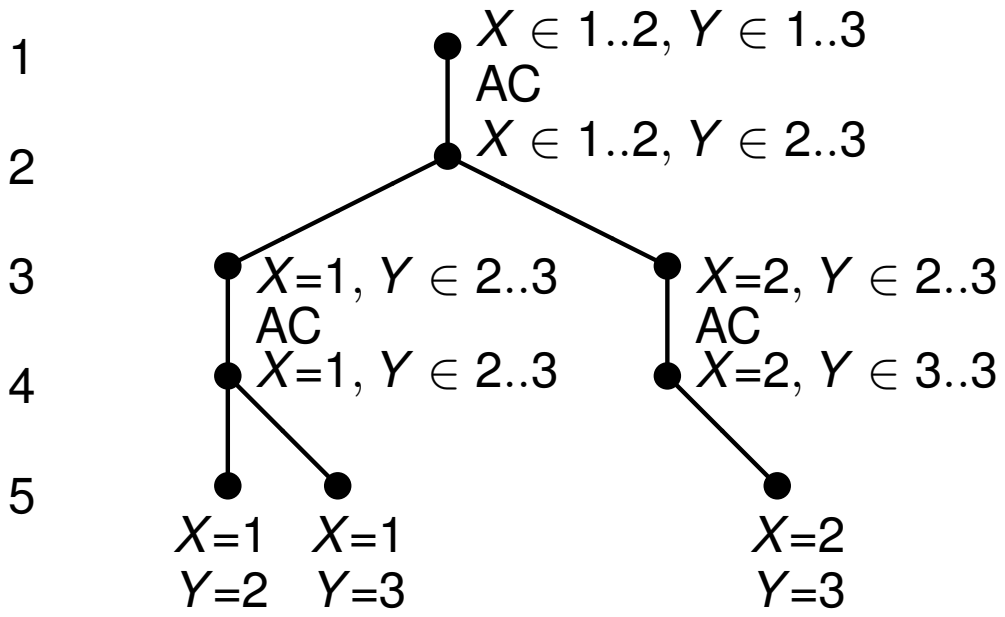
$$\frac{(C_1 \rightarrow C_2)}{\neg C_1 | C_2}$$

2. Absolute value:

$$\frac{|e| = X}{X = e | X = -e}$$

3. Disequality:

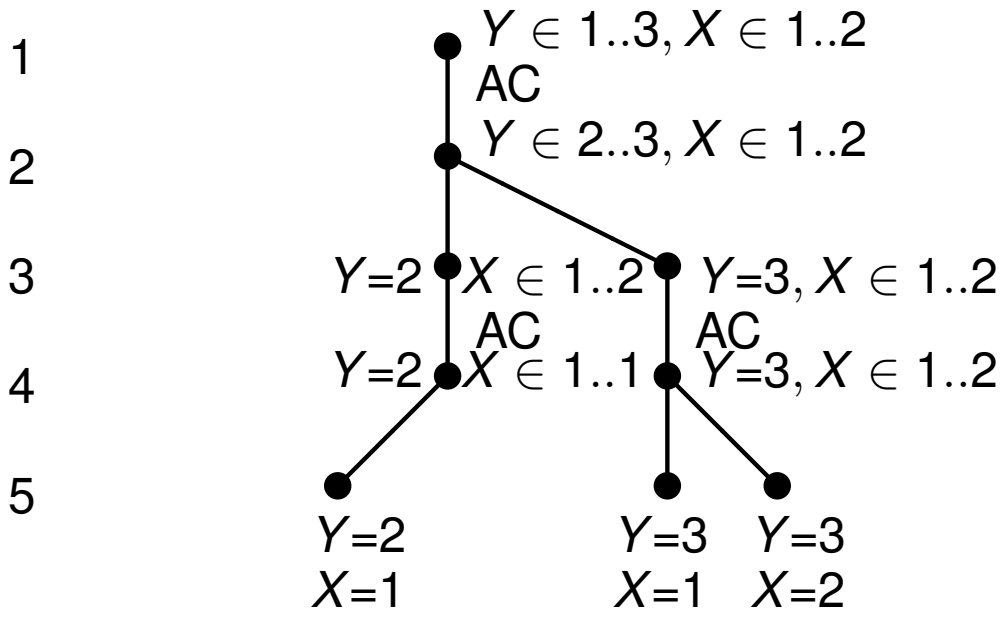
$$\frac{e_1 \neq e_2}{e_1 < e_2 | e_2 < e_1}$$



prop-labeling-tree for $\mathcal{P} = \langle X < Y; X \in \{1,2\}, Y \in \{1,2,3\}\rangle$.

Constraint Solving

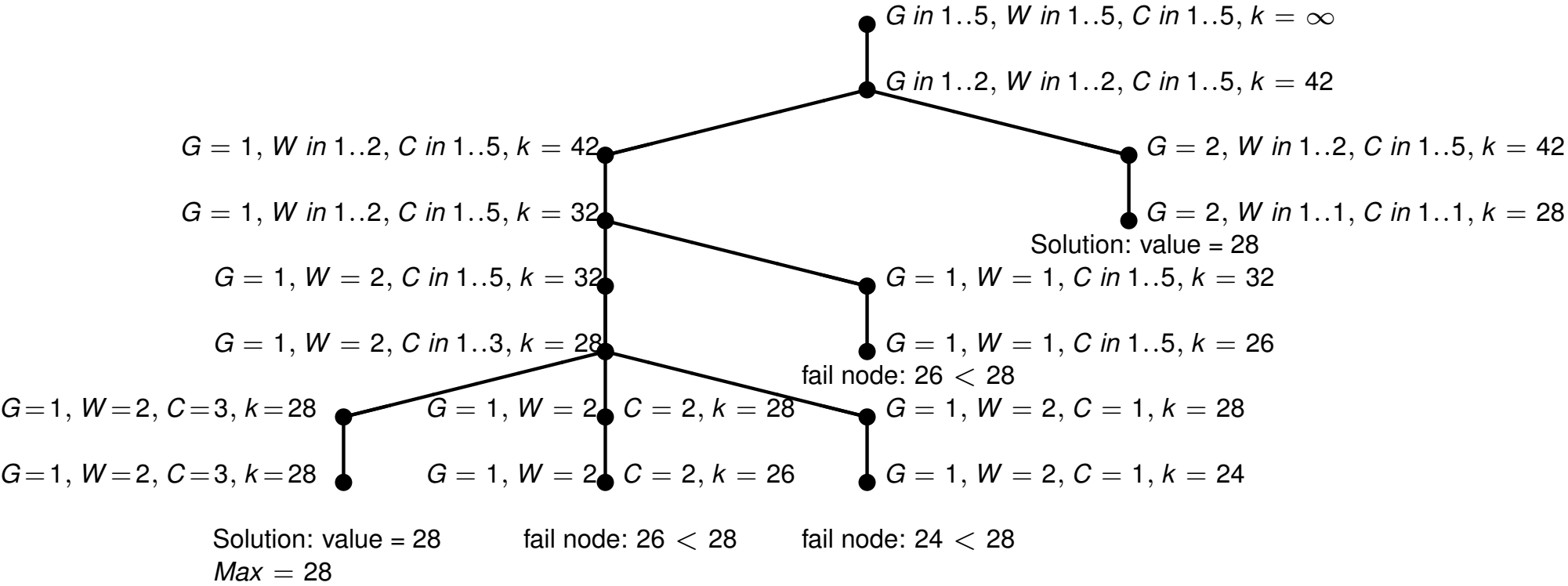
prop-labeling-tree (Y first)



prop-labeling-tree for $\mathcal{P} = \langle X < Y; X \in \{1,2\}, Y \in \{1,2,3\}\rangle$.

- The construction and the visit of the prop-labeling-tree is called by calling a built-in (`labeling` in CLPFD, `solve` in Minizinc).
- Every constraint solver offers a set of parameters:
- Variable choice algorithm (leftmost, ff, ffc, . . .)
- Value in the domain to be enumerated(min, max, med, . . .)
- Other parameters (approximated search, LNS, timeouts, . . .)

Constraint Solving

Example: Branch and Bound for COP

$\mathcal{C} = 17G + 10W + 4C < 50, f(W, G, C) = 10G + 6W + 2C$